Bayes Nets

CSE 537 Project 03 - Questions 1-4 and performance analysis

October 29, 2026

1. Method and measurement

Only the four designated implementation sections in inference.py and factorOperations.py are changed. All 18 supplied support, layout, and example-test files remain unchanged. Function signatures, constants, type checks, wrappers, tracking code, and original Berkeley licensing notices are preserved.

Correctness is checked with the four supplied examples and an independent exhaustive oracle. Performance measurements use Python 3.12.5 and synthetic binary star networks with seed 537. PYTHONHASHSEED=1234 fixes set iteration across runs. Every method gets one warmup; runtime is the median of 7 untraced wall-clock queries, and peak Python allocation is the median of 5 separate tracemalloc queries. Model construction and oracle checks are excluded; inference, evidence specialization, and normalization are included.

2. Question 1: Bayes net structure

Create Pacman, Ghost0, Ghost1, Observation0, and Observation1. Observation0 has parents Pacman and Ghost0; Observation1 has parents Pacman and Ghost1. Agent domains include every grid cell, including walls. The maximum true Manhattan distance is (W - 1) + (H - 1); adding the noise bound of 7 gives an inclusive observation range of 0 through W + H + 5.

Case	Agent domain size	Observation domain
Supplied 4 x 3 grid	12 positions each	0 through 12
Single-cell 1 x 1 grid	1 position each	0 through 7
Additional boundary grids	1 x 8, 9 x 1, 2 x 7, 8 x 5	All exact bounds pass

Table 1. Q1 graph and domain verification. All four edges and all five variables are checked.

3. Question 2: Join factors

The output unconditioned set is the union of input unconditioned sets. The conditioned set is the union of conditioned sets minus all output unconditioned variables. Construct a new factor with the complete domain dictionary, then multiply matching input entries for each output assignment. General factors need not be normalized, so this operation performs no normalization.

Badge / Alarm	Expected product	Result
green / off	0.4 x 0.15 = 0.06	PASS
green / on	0.4 x 0.85 = 0.34	PASS
gold / off	0.6 x 0.8 = 0.48	PASS
gold / on	0.6 x 0.2 = 0.12	PASS

Table 2. Supplied Q2 example; additional cases cover unnormalized values, zeros, general scopes, disconnected inputs, iterator input, and a single factor.

4. Question 3: Eliminate a variable

Remove the requested variable from the unconditioned set, preserving the conditioned set and complete domain dictionary. For each output assignment, extend a copy with each possible eliminated value and sum the input entries. The source factor is unchanged. Supplied guards still reject a missing or conditioned variable and elimination of the only unconditioned variable.

Zone / Badge	Expected marginal	Result
quiet / green	0.10 + 0.25 = 0.35	PASS
quiet / gold	0.40 + 0.25 = 0.65	PASS
busy / green	0.30 + 0.25 = 0.55	PASS
busy / gold	0.20 + 0.25 = 0.45	PASS

Table 3. Supplied Q3 example. Additional tests cover four-value and singleton domains, immutability, unused domains, and exception/tracking behavior.

5. Question 4: Variable elimination

Specialize CPTs using the evidence. In the specified hidden-variable order, join all factors containing the next variable using the tracked helper, then eliminate that variable immediately. Discard a joined factor with only one unconditioned variable, as specified by the assignment. Join the remaining factors and use the supplied normalize helper to return a conditional distribution. Zero-total evidence returns None; singleton-domain variables become conditioned.

Fuel given Lamp=lit	Unnormalized mass	Posterior
full	0.3 x (0.8 x 0.9 + 0.2 x 0.1) = 0.222	111/202 = 0.549505
low	0.7 x (0.2 x 0.9 + 0.8 x 0.1) = 0.182	91/202 = 0.450495

Table 4. Supplied Q4 example. The evidence probability is 0.404; both posterior entries pass.

6. Verification

All four supplied examples and all 16 additional test methods pass. The randomized method checks **1,155 posterior queries across 100 seeded random DAGs**, each with 2-6 binary or ternary variables. It checks one- or two-variable queries, zero to two evidence variables, every hidden-variable permutation, and the default sorted order.

The independent oracle enumerates complete assignments, multiplies original CPT entries, sums query masses consistent with the evidence, and divides by evidence probability. It calls neither implemented factor operation nor either inference routine. Comparisons use relative tolerance 1e-10 and absolute tolerance 1e-12. The suite also verifies factor scopes, full domain dictionaries, evidence restriction, unit total mass, and unchanged input CPTs.

Targeted tests cover exact join/eliminate traces, the discard branch, disconnected networks, no evidence, multiple query variables, no hidden variables, singleton queries, and impossible evidence. The completed ZIP is extracted to a fresh directory and both suites are rerun successfully. Eight complete engine games also pass across all four layouts: four use stationary ghosts with scripted input through the keyboard agent and shading pipeline; four use random ghosts with a test-only driver that can see ghost positions. Every game captures all ghosts and finishes. The actual GUI was launched and arrow-key and WASD movement, sensor readings, and shading were visually verified. These game checks are separate from Q1-Q4.

From bayesnets/, run `python3 test_q1to4.py` and `python3 -m unittest test_reference_solution test_game_demo -v`. No external packages are needed for the code, tests, or benchmarks.

7. Inference-efficiency comparison

In each binary star, Center is the parent of every Leaf. Query Leaf0 without evidence. The center-first order eliminates Center before the hidden leaves; the leaves-first order eliminates the hidden leaves before Center. Every timed and memory-traced result is checked against the independent oracle.

Variables	Method	Time (ms)	Peak (KiB)
5	VE: center first	0.708	35.05
5	VE: leaves first	0.281	19.75
5	Enumeration	1.086	42.00
7	VE: center first	2.635	85.11
7	VE: leaves first	0.421	25.20
7	Enumeration	5.449	96.46
9	VE: center first	11.840	264.70
9	VE: leaves first	0.576	32.08
9	Enumeration	29.610	337.52

Table 5. Median runtime from 7 untraced queries and median peak Python allocation from 5 separate tracemalloc queries per case.

Seven-variable method	Largest factor rows	Total created rows
VE: center first	128	382
VE: leaves first	4	30
Enumeration	128	1152

Table 6. Separate constructor instrumentation counts newly created factor-table rows, including normalization. These counts are not whole-process memory measurements.

Interpretation and critical analysis

For seven variables, leaves-first elimination reduces the largest created factor from 128 rows to 4, a 32-fold difference, while preserving the posterior (0.3444444444, 0.6555555556). Marginalizing a hidden leaf discards its normalized CPT; eliminating Center first creates a factor coupling all leaves. At nine variables, leaves-first inference is about 51.4 times faster than enumeration in this run.

Variable elimination is exact up to floating-point rounding, but cost can still grow exponentially with the largest intermediate scope. The provided sorted default order is deterministic rather than a performance heuristic. Graph-based choices such as min-fill can reduce intermediate factors, but this implementation correctly honors the requested order. Evidence can shrink domains before any joins.

Limits and reproducibility

These sparse synthetic models demonstrate order sensitivity; they do not establish performance on dense networks. Runtime depends on the machine, Python version, and iteration order. Tracemalloc measures query-time Python allocations, not whole-process memory; table rows are another distinct measure. Extremely small probabilities can underflow. The supplied private grading suite was unavailable, so no hidden-grader score is claimed.

Reproduce with `PYTHONHASHSEED=1234 python3 benchmark.py` and `PYTHONHASHSEED=1234 python3 analyze_order.py`. The JSON files retain full-precision measurements. Based on the supplied CSE 537 handout and Berkeley Pacman code (http://ai.berkeley.edu); retain original notices and keep the TA reference private.